C++ Linux 开发面试前一天速刷：常用调试手段

> **目标导向**：按“一天能刷完、面试前够用”整理。目标不是覆盖所有细节，而是让你在面试里能把常用调试手段、思路、工具、典型问题说清楚。针对 **1~3年经验**，展现出有实际排查经验、思路清晰即可。

一、 先记住面试官最爱听的“调试框架”

回答调试问题时，尽量按以下顺序展开，展现系统性思维：

1. **先稳定复现**
   - 能不能稳定复现？
   - 复现条件是什么：输入、并发、机器环境、编译选项、数据规模。
   - 是否和时间、线程、网络、负载有关？
2. **缩小范围**
   - 最近改了什么？
   - 是代码问题、配置问题、依赖问题，还是环境问题？
   - 是启动即崩、运行一段时间崩，还是高并发下偶发？
3. **看日志 + 看现场**
   - 先看日志定位模块。
   - 崩溃就看 `core dump` / `backtrace` 。
   - 卡死就看线程栈、锁等待、系统调用阻塞点。
4. **用工具验证猜测**
   - `gdb` ：看调用栈、变量、线程。
   - `valgrind` / `sanitizers` ：查内存问题。
   - `strace` ：看系统调用。
   - `top` / `ps` / `pmap` / `lsof` ：看资源。
   - `perf` ：看热点。
5. **修复后回归验证**
   - 不只看“问题没了”，还要验证有没有引入新问题。
   - 增加日志、断言、单测、压测。

> 💡 **这套话术很适合作为以下问题的开场白**：“线上崩溃你怎么查？”、“程序卡死怎么定位？”、“内存泄漏怎么查？”、“CPU 飙高怎么查？”

二、 最常见的 8 类问题和对应手段

1. 程序崩溃（Segmentation Fault / Abort）

- **常见原因**：空指针、野指针、数组/vector越界、use-after-free、栈溢出、double free、assert 触发、异常未捕获。
- **常用手段**：
  1. **先开 core dump**： `ulimit -c unlimited`
  2. **用 gdb 看 core**：

```
gdb ./app core.xxx
(gdb) bt          # 或 bt full
(gdb) frame 0     # 切换到崩溃帧
(gdb) info locals # 看局部变量
(gdb) p variable  # 打印关键变量
```

  3. **如果没有 core，直接运行调试**： `gdb ./app -> run -> bt`

  🗣️ **面试表达**：“如果程序崩溃，我会优先拿 core 文件，用 gdb 看 backtrace，先确认崩溃点函数、调用链和关键变量。如果符号不全，我会检查是否带了调试符号。然后结合日志、输入数据和最近改动缩小范围。如果怀疑是内存踩踏，我会进一步用 AddressSanitizer 或 valgrind 验证。”

2. 程序卡死 / 无响应

- **常见原因**：死锁、条件变量没唤醒、死循环、IO 阻塞、网络阻塞、futex 等待、某个线程 hold 锁太久。
- **常用手段**：
  1. **先看进程状态**： `top -H -p <pid>` 或 `ps -T -p <pid>`
  2. **gdb attach 看所有线程栈（⚠️ 高频必备）**：

```
gdb -p <pid>
(gdb) info threads
(gdb) thread apply all bt  # 极其重要
```

  3. **看系统调用阻塞**： `strace -p <pid>`
     - *卡在 read/write*：可能 IO 卡住
     - *卡在 futex*：多半是锁竞争/死锁
     - *卡在 epoll_wait*：可能在等事件，未必是异常
     - *卡在 nanosleep*：可能在 sleep

  🗣️ **面试表达**：“卡死问题我一般先区分是业务‘慢’还是进程‘挂住’。如果是进程无响应，我会先用 gdb attach，执行 `thread apply all bt` 查看所有线程栈，判断是否死锁、锁等待、死循环或系统调用阻塞。必要时配合 strace 看当前阻塞在哪个系统调用上。”

3. 内存泄漏

- **常见原因**：new/malloc 后没释放、 `shared_ptr` 循环引用、容器持续增长、缓存设计不当。
- **常用手段**：
  1. **Valgrind**： `valgrind --leak-check=full --show-leak-kinds=all ./app`
  2. **看进程内存变化**： `top` 、 `pmap -x <pid>` 、 `cat /proc/<pid>/status` （重点关注 `VmRSS` , `VmSize` ）
  3. **排查 shared_ptr 循环引用**：A 持有 B 的 shared_ptr，B 持有 A 的 shared_ptr -> 用 `weak_ptr` 打破环。

  面试表达：“内存泄漏我会先区分是真泄漏还是业务缓存增长。如果是测试环境可复现，我优先用 valgrind。在线上会结合 RSS、对象数量、缓存命中率和生命周期分析。如果使用智能指针，也会特别检查 shared_ptr 循环引用问题。”

4. 内存非法访问 / 越界 / UAF

这类问题比普通泄漏更高频，也更致命。**最好用的手段是 Sanitizer。**

- **编译时加**： `-g -O1 -fsanitize=address -fno-omit-frame-pointer`
- **运行后直接报错**： `heap-buffer-overflow` , `stack-buffer-overflow` , `use-after-free` , `double free`
- 查数据竞争： `-fsanitize=thread`
- 查未定义行为： `-fsanitize=undefined`

  面试表达：“对于越界、use-after-free、double free 这类问题，我更倾向先用 AddressSanitizer，因为它比纯手工看代码效率高，报错也更直接。多线程竞争问题会考虑 ThreadSanitizer。”

5. CPU 飙高

- **常见原因**：死循环、自旋锁竞争、某段算法复杂度过高、频繁重试、日志刷太猛。
- **常用手段**：
  1. **先看哪个线程占 CPU**： `top -H -p <pid>`
  2. **抓热点**： `perf top -p <pid>` 或 `perf record -p <pid> + perf report`
  3. **gdb 看高 CPU 线程栈**：从 top 找线程 ID，转成 gdb 线程查看。

  面试表达：“CPU 飙高我通常先定位到具体线程，再区分是业务热点还是异常空转。如果是热点分析，我会用 perf；如果怀疑死循环或自旋，会配合 gdb 看线程栈和循环变量。”

6. 多线程问题

- **常见类型**：死锁、数据竞争、锁顺序不一致、条件变量误用、原子变量使用不当。
- **面试加分点（主动提）**：
  - 看线程栈： `thread apply all bt` ，看是否都卡在 `futex` 。
  - 检查加锁顺序是否统一，尽量缩小锁粒度。

- 能用 RAII 锁对象（`std::lock_guard`，`std::unique_lock`），绝不手动 lock/unlock。
- 数据竞争优先怀疑共享变量。
- 明确“原子变量不等于线程安全全部解决”。
- 避免在持锁状态下做耗时操作。

7. IO / 网络 / 文件句柄问题

- **常见问题**：文件打开失败、fd 泄漏、网络超时、连接数耗尽、epoll 使用错误。
- **常用手段**：
  - **查看 fd**：`lsof -p <pid>` 或 `ls /proc/<pid>/fd | wc -l`
  - **看网络连接**：`ss -antp` 或 `netstat -antp`
  - **看系统调用**：`strace -p <pid>`

  🗣 **面试表达**：“对 IO 或网络问题，我一般会从 fd、连接状态、系统调用和超时机制入手。如果怀疑 fd 泄漏，会看 /proc/pid/fd 和 lsof；如果怀疑阻塞或超时，就用 strace 看 read/write/connect/epoll_wait。”

8. 线上问题排查

- **标准答法：先止血，再定位，再复盘。**
  - **止血**：回滚版本、降级、限流、重启异常实例、熔断某些功能。
  - **定位**：看监控（CPU/内存/QPS/报错）、看日志、保留现场（core/线程栈/配置）。
  - **复盘**：补日志、补告警、补测试、补防御式编程。

  🗣 **面试表达**：“线上问题我不会直接先改代码，而是优先保证服务可用，比如回滚或重启异常实例。之后再保留现场，通过日志、监控、core 文件、线程栈来定位。最后会补上防复发措施。”

三、 必须掌握的 Linux / C++ 调试工具命令

1. gdb —— 最核心

- **常用命令**：`run`，`break`，`next`，`step`，`continue`，`print var`，`bt full`，`info locals`，`info args`，`info threads`，`thread apply all bt`，`watch var`
- **面试怎么说**：
  - 崩溃：`bt`
  - 多线程卡死：`info threads` + `thread apply all bt`
  - attach 在线进程：`gdb -p pid`

2. Valgrind & Sanitizer

- **Valgrind**：查泄漏、非法访问。缺点是太慢，对线上不友好。 `valgrind --leak-check=full ./app`

- **Sanitizer（必背）**：对开发/测试阶段极好用，定位极快。 `-fsanitize=address` , `-fsanitize=thread` , `-fsanitize=undefined`

3. strace & perf

- **strace**：看卡在哪个系统调用、文件/网络操作是否失败、errno 是什么。 `strace -p <pid>`
- **perf**：查 CPU 热点、看函数耗时分布。 `perf top -p <pid>`

4. 常用 Linux 状态查看命令

- `top` / `top -H -p <pid>`
- `ps -ef` / `ps -T -p <pid>`
- `pmap -x <pid>`
- `lsof -p <pid>`
- `ss -antp`

四、 面试高频：真实问题案例（建议背诵2个）

- **案例 1：空指针崩溃**

  > “有次服务启动后偶发崩溃，先通过 core + gdb 看 backtrace，发现崩在一个对象方法调用上。继续看变量发现该对象在某个异常分支下没有初始化。最后补了空指针判断，并调整初始化顺序，同时补了启动流程日志。”

- **案例 2：死锁**

  > “有次一个接口偶尔超时，gdb attach 后用 `thread apply all bt` 看到两个线程分别持有不同锁并相互等待。根因是两个模块加锁顺序不一致。后来统一了锁顺序，并缩小了锁作用范围。”

- **案例 3：内存泄漏**

  > “某后台进程运行几天后内存持续上涨，先看 RSS 确实在增长。用 valgrind 和代码审查发现是一个缓存 map 只有插入没有淘汰，另外有一处 shared_ptr 循环引用。之后加了淘汰策略，并把一端改成 weak_ptr。”

五、 一天速记版：必须背下来的 8 个命令组合

1. **崩溃开 Core：** `ulimit -c unlimited`
2. **看崩溃栈：** `gdb ./app core.xxx -> bt full`
3. **在线进程卡死：** `gdb -p <pid> -> thread apply all bt`
4. **看系统调用阻塞：** `strace -p <pid>`
5. **看线程 CPU 占用：** `top -H -p <pid>`
6. **查内存泄漏：** `valgrind --leak-check=full ./app`
7. **查 CPU 热点：** `perf top -p <pid>`

8. **看句柄泄露：** `lsof -p <pid>`

六、 面试前最后记住的 10 句话（排查心法）

1. 先复现，再定位，再修复，再验证。
2. 崩溃优先看 core 和 backtrace。
3. 卡死优先看所有线程栈。
4. 内存问题优先想到 ASan / valgrind。
5. CPU 高先定位线程，再看 perf。
6. strace 适合看系统调用阻塞。
7. 共享指针要警惕循环引用。
8. 多线程问题先怀疑锁顺序和共享变量。
9. 线上问题先止血，再保留现场。
10. 修复后一定做回归验证。

保持自信！不需要把自己包装成“底层专家”，只要把 `gdb`，`core dump`，`thread apply all bt`，`ASan`，`strace`，`perf` 这几个基本盘讲清楚，展现出清晰的排查思路，对于 1~3 年经验的候选人已经非常有说服力了。